FOR COMPLETE BEGINNERS

cannot find symbol?

Complete Modern Java, episode 4: read your first errors in Java and TypeScript

> An error is a clue, not a failure. Read the message, find the line, change one thing, run again.

We broke the grocery receipt on purpose, in Java and in TypeScript, and read every message it produced.

Where an error appears tells you which kind you have: before the program runs, while it runs, or never.

In this episode

- Syntax and compile errors
- Runtime errors and the stack trace
- Logic errors: no message at all
- One habit for every error

Coming next

- The debugger (episode 5)
- try and catch (a later module)
- Tests (later in the course)

At the time of recording: Java 25, Node 24 LTS, TypeScript 7.0.2.

01 Four kinds of error

Syntax, Java	`error: ';' expected. Nothing runs.`
Syntax, TypeScript	`TS1005: ')' expected. Node refuses too.`
Compile, Java	`cannot find symbol, then symbol: and location:`
Compile, TypeScript	`TS2551 ... Did you mean 'toUpperCase'? Node runs it and crashes.`
Runtime, Java	`NumberFormatException: For input string: "one"`
Runtime, TypeScript	`No crash: Milk: NaN`
Logic, both	`Total: $4.99, no message at all`

Each one broken, read, fixed and run again, in IntelliJ's terminal.

02 Read the parts of a message

The file, the line, in TypeScript the column and an error code, then the message in words and the line itself with the spot marked.

The location is where the compiler noticed the problem, not always where it was made. If the line it names looks fine, look at the line above.

java src/GroceryReceipt.java

```
src/GroceryReceipt.java:2: error: ';' expected
    int bread = 2
                ^
1 error
error: compilation failed
```

> tsc reported the missing) as groceryReceipt.ts:9:39 - error TS1005: ')' expected.
>
> Run on a Mac, 2026-10-23: Java 25.0.4.1, Node 24.22.1, TypeScript 7.0.2.

03 Check first, then run

Node does not read types. It runs the file and finds a problem only when it reaches it. The compiler reads the whole file first.

typo.ts

```
const customer: string = "Sarah";
console.log(customer.toUppercase() + "'s groceries");

// npx tsc --noEmit typo.ts
// TS2551: ... Did you mean 'toUpperCase'?
// node typo.ts
// TypeError: customer.toUppercase is not a function
```

> Java reports the same typo before running: cannot find symbol, symbol: method toUppercase(), location: variable customer of type String.
>
> Run on a Mac, 2026-10-23: Java 25.0.4.1, Node 24.22.1, TypeScript 7.0.2.

04 Read a stack trace

First the top line: the exception and its message. Then the first line from your own code. The lines above it are Java's own code, the path the problem took.

java src/Runtime.java

```
Exception in thread "main" java.lang.NumberFormatException: For input string: "one"
    at java.base/java.lang.NumberFormatException.forInputString(...)
    at java.base/java.lang.Integer.parseInt(Integer.java:565)
    at java.base/java.lang.Integer.parseInt(Integer.java:662)
    at Runtime.main(Runtime.java:3)    <- start here
```

> Number("one") in TypeScript does not crash: it is NaN, and the program prints Milk: NaN.
>
> Run on a Mac, 2026-10-23: Java 25.0.4.1, Node 24.22.1, TypeScript 7.0.2.

05 Predict before you run

A logic error has no message. The compiler cannot know what the receipt should total. Work it out first: 2 x $1.50 + 1 x $0.99 = $3.99.

one character

```
double total = bread * breadPrice + milk * milkPrice;   // $3.99
double total = bread * breadPrice + milk + milkPrice;   // $4.99, no error
```

> Both languages printed Total: $4.99 with exit code 0, and tsc --noEmit was quiet.
>
> Run on a Mac, 2026-10-23: Java 25.0.4.1, Node 24.22.1, TypeScript 7.0.2.

One habit for every error

1. Read the main message: the error name, and its words.
2. Find the location. In a stack trace, the first line from your code.
3. Open that line, and the line above it.
4. Change one thing, and run it again.
5. Challenge: fix BrokenReceipt.java and brokenReceipt.ts until both print Total: $3.99.

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-23 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.